PFL - Prolog Coursework

Topic and Group

Group Name: Aquapipe_3

Group Members

- António Lino dos Santos (201705558) - 30% Contribution
- Gabriel Tomaz Machado Júnior (202008860) - 35% Contribution
- Manuel Rivera Villatte (202401168) - 35% Contribution

Installation and Execution

Our Application runs much like any other designed in class. Just dropping the files into a SICStus Prolog 4.9.0 enviromemnt and calling the `play/0` predicate will start the application.

Topic (Game) and Rules

The topic, as stated in the group name, is Aqua Pipe.
In this game, the player's mission is to make aqua pipe lines.

AquaPipe 3-in-a-row (3x3 Board)

- Players: 2-players
- Board: 3x3 spaces
- Pieces: 3 straight pipes of each diameter (3 x 12mm, 3 x 16mm, 3 x 19mm) for each player, red and blue.

Winning Conditions

- By making a row of 3 pipes with the same diameter and same color, horizontally, vertically or diagonally.

Rules At the beginning of the game, there are no pieces on the board. The players choose the colors and who is the first player by casting dice or coin toss.

During the game, the following rules apply:

- Players may place one of their own pipes at any available space on the board or move one of their own pipes to another available space.
- Player can only move a pipe on the board after he/she placed at least one each of the three size pipes.
- Three different size pipes can be placed on the same space because of their structure.
- **Optional Rule**: Each player can only move pieces in the board after placing all of his pieces in it (only in optional rule mode)

Sources of Game Rules Game Page on Kickstarter
Game Page on BoardGameGeek

Note: The game also has a 4x4 mode, which we tried to implement, but we due to complications and time constraints and couldn't implement it.

Considerations for game extensions

We chose to add an additional rule to the base game that is explained in more detail in the rules section. The rule was considered to restrict the mobility of placed pipes in boards.

Game Logic

Game Configuration Representation

Mode-Players-Level where,

- *Mode* - One of the 2 Game Modes avaliable: 3x3 or 3x3-O (Optional Rule)
- *Players (or F-CF/S-CS)* - Can be h-blue/h-red, h-blue/pc-red, pc-blue/h-red, pc-blue/pc-red,
  where h -> Human, pc -> Computer and blue/red is the color of the pieces of a player
- *Level* - represents the level of the PC, it can be Random, Greedy or Minimax

Internal Game State Representation

Mode-F-CF/S-CS-Level-Board-P-CP-PossibleMoves where,

- *Mode* - One of the 2 Game Modes avaliable: 3x3 or 3x3-O (Optional Rule)
- *F & CF* - First Player, F (h or pc) with color blue (CF - Color F)
- *S & CS* - Second Player, S (h or pc) with color red (CS - Color S)
- *Level* - represents the level of the PC, it can be Random, Greedy or Minimax
- *Board* - Bi-dimensional list of 3x3 size
- *P* - Player to play on the current turn (F on the first turn)
- *CP* - Color of player P
- *PossibleMoves* - list with the moves that can be made by P on the current game state

Piece Representation

CurrentPlayer-PlayerColor-PipeType-PipeNumber-InBoard-RowInBoard/ColInBoard where,

- *CurrentPlayer*: player (h or pc).
- *PlayerColor*: color of the CurrentPlayer (blue or red)

- *Pipe (PipeType/PipeNumber)*: type of pipe that was placed/moved (s, m, l) and its index (1, 2, 3)
- *InBoard*: boolean value that represents if piece is in board or not, false meaning out of board, true meaning in the board
- *RowInBoard & ColInBoard*: If piece is in board, represents the row and column where it is, otherwise n/n

Move Representation

Piece-DRow/DCol-DRowUPipe/DColUpipe where,

- *Piece*: represents the piece to be moved
- *DRow & DCol*: destination position in the board where the piece will be placed.

Minimax Algorithm Strategy

The algorithm was designed with the recursive `minimax` function that has a base case and a recursive case.

The idea is that on every turn, the player can be either the Maximizing Player or the Minimizing Player.

The Minimax PC is going to be the Maximizing, and its opponent the Minimizing.

The algorithm looks for valid moves, then iterates over them, applying each one and then recursively calling `minimax`. When a terminal node is found (or depth becomes 0), then the algorithm calculates the `value` for the given GameState and returns it. When there are no moves left to analyze, the algorithm either looks for the move that yielded the minimum or the maximum value, depending on whichever player performed the move.

The algorithm recursively goes through the whole tree. It is worth noting that for a depth of only 2, the pc takes quite a while to perform a move. Try with depths 3 and above with discretion. We are not to be held liable for any memory overflows. (we have lawyers.)

For any doubts, the code itself is thoroughly documented (well, as thoroughly as we could).

Notes

1. Due to misattention from the beginning and lack of time, we weren't able to make the coordinates start at (1,1) at the lower left corner.

Conclusions

The project was developed innitially without the option of moving pipes once they were already placed, and that feature was only implemented later. A similar strategy was used for the development of the 4x4 board and bridge pipe rules, but, due to issues with pipe validation and struggles with the debugging process, the 4x4 mode was not ready in due time. An additional process was developing a minimax algorithm that would proccess its move in a short time, as our current model takes a long time to calculate its next move. In future development, we could finish implementing the features we had to cut, such as the 4x4 mode, as well as improving our computer minimax algorithm to run faster and/or better. We could also allow for players to quit the application in the middle of a game through menus, and an option to save a midway game. We could also add stat tracking and a leaderboard for player comparison.

Bibliography

- Documentation for SICStus Prolog 4.9.0
- PFL course lecture resources